Software Requirements Specification

Project Title: Veritas
Version 0
Group 19
COMPSCI 4ZP6
Supervisor: Dr. Allison Lahnala
October 10, 2025

Haneef Taher	tahers7@mcmaster.ca	400359025	Claim Verification Model Lead Developer
Keyu Lin	link55@mcmaster.ca	400392492	Program Manager, Claim Verification Model Developer
Madhav Bezawada	bezawadm@mcmaster.ca	400380258	Claim Verification Model Developer
Divij Dhiraaj	dhiraajd@mcmaster.ca	400360225	Bias Detection Model Developer
Louis Doan	doanl7@mcmaster.ca	400395183	Front-End Developer
Mustakim Kazi	kazim18@mcmaster.ca	400390604	Bias Detection Model Lead Developer
Dr. Allison Lahnala	lahnalaa@mcmaster.ca		Supervisor

Table of Contents

Document Information **3**
Glossary 3
Contributions 4
1. The Purpose of the Project **4**
2. The Client and other Stakeholders **5**
Primary Audience 5
The Stakeholders 5
3. Project Constraints **5**
3.1 ML Model Constraints 5
3.2 Website and Extension Constraints 6
4. Functional Requirements **6**
P0 6
P1 6
P2 7
P3 7
5. Data and Metrics **7**
1. Data sources 7
2. Evaluation Metrics 7
6. Non-functional Requirements **8**
1. Look and Feel requirements 8
2. Usability and Humanity requirements 8
3. Performance and speed requirements 9
4. Security and Privacy 9
5. Legal 9
7. Potential Risks and Issues **9**
1. Model accuracy 9
2. Quality of data for training the model 9
3. Usage of ongoing events 10
4. Usage on sensitive topics 10
5. Transparency 10
6. Evaluation 10
7. Wide topics covered 10
8. Team Meeting and Communication **12**
9. Team Member Roles **12**

9.1 Team Roles Overview..... 12
9.2 Role Descriptions..... 12
10. Workflow Plan..... 13
11. Proof of Concept Demonstration Plan..... 14
12. Technology.....15
Frontend..... 15
Web Extension..... 15
Backend.....15
Infrastructure..... 15
Machine Learning..... 15
Development Workflow Tools.....16
13. Project Scheduling (Gantt Chart):..... 16

Document Information

Glossary

Fake News	News that contains objectively false information.
ML	An acronym for machine learning.
Bias	Refers to political bias. Can be “left”, “center”, and “right”.
FEVER Score	A metric that evaluates fact-checking systems by requiring both the correct claim label and the correct supporting evidence.
F1	The F1 score is the harmonic mean of precision and recall.
Macro-F1	Average F1 score for each class separately.
Recall@k	Measures the frequency of correct evidence appearing within the top k retrieved results.

Contributions

Authors	**Sections**
Haneef Taher	Functional Requirements, Team Meeting and Communication Plan
Keyu Lin	Project Constraints, Workflow Plan
Madhav Bezawada	Functional Requirements, Proof of Concept Demonstration Plan
Divij Dhiraaj	The Purpose of the Project, The Client and Other Stakeholders
Louis Doan	Data and Metrics, Non-functional requirements, Risks and issues predicted, Technology
Mustakim Kazi	Data and Metrics, Non-functional requirements, Risks and issues predicted, Project Scheduling (Gantt Chart)

1. The Purpose of the Project

The purpose of the project is to give users a tool to fact-check content that they encounter in day-to-day life, such as an article or a passage of text. The system evaluates individual claims on a spectrum - supported, refuted, and not enough evidence. In addition, it determines the political or ideological leaning of the overall content (left-leaning, right-leaning or centrist). Once this leaning is identified, we provide an abstractive summary of the evidence used for refuting claims as a means to explore media framing.

2. The Client and other Stakeholders

Primary Audience

The primary audience for this product is individuals who are skeptical of ideas or principles presented in articles or media. These users seek tools that enable them to:

1. Fact-check the accuracy of content.
2. Identify potential bias or ideological leaning (left, right, or centrist).
3. Explore counter-evidence that challenges claims presented by popular outlets.

The Stakeholders

1. Individuals:
    1.1. People fact-checking and open to claim-level counterpoints grounded in evidence.
2. Verification Resources:
    2.1. Verification resources interested in using our product as a fact checking tool
3. Course Stakeholders:
    3.1. Project supervisors and teaching staff who oversee the development of the system.

3. Project Constraints

3.1 ML Model Constraints

We are planning on incorporating multiple NLP models into our product, which are highly complex and hard to fine-tune. We could run into the classic ML problems of whether the data is complete, accurate, representative, and relevant. We can only work with the data available and cannot extract and perform our own data collection.

The list of possible NLP models in our product includes:

1. Claim extraction/verification
2. Bias detection
3. Counterpoint generation

Training and storing this data must be feasible, either through our local processing power or by utilizing CAS resources. There is 1TB of available storage for relevant datasets and model-related materials. However, CAS resources and constraints are still unknown, as we await a response from the department regarding our allotment.

3.2 Website and Extension Constraints

We are planning on creating our product on a browser medium as a web extension, along with a website to support documentation, download, and details of the extension.

The website will be hosted on a cloud server or someone's local device, which may incur hosting and domain costs. Our hope is that these do not surpass $30/month and will be manageable for demo and the purposes of this course. The website is subject to accessibility constraints such as colourblindness support, partial or full blindness support, and inclusive peripheral usage. We will not be able to incorporate accessibility practices for all scenarios.

As for our extension medium, we assume and constrain the scope of this project to only Chromium browsers, as developing support for cross-browser support is extensive, and most of the popular browsers and most users have access to/use a Chromium browser.

As articles or pieces of text are submitted through our browser extension, we assume there should be a way to read and parse the content submitted. If the submitted contents are not parsable, the product cannot process and produce its expected output.

4. Functional Requirements

P0

- A chromium browser extension that takes in an excerpt of text or a whole article and detects claims for selected text/article, analyzes the bias, sends to the backend, and renders the verdict inline.
- Landing page for install, privacy policy, and status.
- Evidence retrieval from reputable sources with claim-level Supported/Refuted/Insufficient labels.
- Basic bias signal for the source of the original content and referenced material used for the confidence score.
- Display of cited evidence snippets and sources on hover or side panel.
- Minimal telemetry and error handling in extension (timeouts, retries, graceful degradation).

P1

- Implement an entailment check for the generated abstract summary of evidence snippets to reduce hallucinations.
- Simple stats dashboard via pop-up using the extension’s local history

- Optimization of ML model inference for faster response times
- Automated testing pipelines for continuous deployment
- User Authentication section on the landing page for user metrics
- Counterpoint generation for extracted claims

P2

- Cross-article claim memory to efficiently answer repeated inputs
- Toggle bar between conservative/liberal/neutral articles that are reporting the same topic
- Community-sourced "truth ratings" on claims
- User-controlled analysis depth (quick scan vs deep-dive report)
- Account-free settings in extension: toggle counterpoints, bias view, and strict citations-only mode.

P3

- Bias trend analysis over time for a given source or author
- Personalized dashboards showing user-specific trends in submissions
- Bulk ingestion API for institutions to analyze hundreds of articles at once
- Analyze the same news story across different languages to detect differences in bias

5. Data and Metrics

We will have two key models, Bias Detection and Claim Verification. Each model will have its own separate training data and evaluation criteria. In addition to our key models, we will also have counterpoints and bias/claim extraction.

1. Data sources
    - 1.1. Bias Detection
        - 1.1.1. BABE - Expert-annotated sentences with bias labels and rationale spans.
        - 1.1.2. QBias - Articles curated for balanced perspectives and bias research.
        - 1.1.3. NewsMediaBias - Multi-dimensional bias labels for auxiliary training and analysis.
            - 1.1.3.1. In addition to the datasets we use, we will ensure that our data has an even split of left, center and right-leaning perspectives.
    - 1.2. Claim Verification
        - 1.2.1. FEVER - Wikipedia-derived claims with labels and evidence sentences for supervised verification.
        - 1.2.2. MultiFC - Multi-domain claims aggregated from fact-checking sites with real-world variability.
        - 1.2.3. AVeriTeC - Real-world claims linked to online evidence and textual justifications.
        - 1.2.4. LIAR - PolitiFact statements with fine-grained truth labels and metadata.

1.2.4.1. In addition to these datasets, we will use existing APIs to retrieve more recent information as needed (e.g., Wikipedia passages, news API integrations, etc).

2. Evaluation Metrics
   2.1. Bias Detection
      2.1.1. Our primary evaluation metric will be a macro-F1 score, with a target of ~75%.
      2.1.2. Additionally, we will also look at per-class precision/recall and a confusion matrix for error analysis.
         2.1.2.1. Macro-F1 ensures balanced performance across all our classes (Left, Center, Right) despite potential class imbalances. At the same time, recall, precision and confusion matrices will help us diagnose which biases are being under or over-predicted.
   2.2. Claim Verification
      2.2.1. Our primary evaluation metric will be the FEVER Score, which determines whether the label and evidence are correct.
      2.2.2. Additionally, we will also look at per-class F1 scores for the claim labels, aiming for ~80% F1.
      2.2.3. We will also look at recall@k for evidence retrieval quality.
         2.2.3.1. Fever score is the standard evaluation metric for fact-checking tasks, ensuring both the label and evidence are correct. Per-class F1 highlights the balance across labels, and recall@k ensures that the evidence retrieved is related and useful to the facts we're trying to verify.

Additionally, for counterpoint generation, we will use human evaluation to determine persuasiveness, diversity of perspectives, and ethical impact.

Training all models will include a standard pre-processing step to normalize article text (removing HTML, handling special characters, converting to lowercase, etc.). We will split our training data into 80/10/10 splits for training, testing and validation. Each team responsible for a model will evaluate its speed and accuracy, aiming to improve these aspects while sharing findings with other teams.

6. Non-functional Requirements

1. Look and Feel requirements
   1.1. The frontend/extension will feature a modern, easy-to-understand design, enabling users to quickly fact-check articles, videos, and other media with just one click.

1.2. The backend API will have clear documentation allowing for easy usage and interfacing with the frontend/extension.
1.3. Any errors will be displayed to the user in straightforward and easy-to-understand terms.
1.4. Output is easily readable in human-friendly language

2. Usability and Humanity requirements
   2.1. The frontend will aim to be accessible to all people regardless of disabilities.
      2.1.1. Will aim to be usable using screen-readers and other assistive technologies
      2.1.2. Will aim to adhere to ARIA labels alongside best practices.
   2.2. The frontend will also be usable on all screen sizes, such as tablets and mobile devices.
   2.3. The model and UI will remain non-discriminatory, ensuring equitable treatment of marginalized groups.
3. Performance and speed requirements
   3.1. The model should be able to generate results within a reasonable time window.
   3.2. Progress indicators will be displayed, showing the user that some actions are being taken.
   3.3. The system should be able to handle multiple user requests concurrently without significant performance degradation.
4. Security and Privacy
   4.1. The frontend/extension's communications will be encrypted using HTTPS
      4.1.1. Communications from the backend API will be secure.
   4.2. User data will be protected in the database, passwords and sensitive information will be hashed and salted before being stored.
   4.3. The data used to train the ML model will be ethically sourced, ensuring that it avoids privacy violations and improper scraping.
5. Legal
   5.1. We will not circumvent paywalled content to scrape articles.
   5.2. The project must not label something incorrectly, as it may be defamatory (e.g. labelling real news as fake)
   5.3. Confidence scores and rationales will be made transparent to the user to support ethical use.

7. Potential Risks and Issues

1. Model accuracy
   1.1. The model has to maintain a high level of accuracy. This is incredibly important in the context of this project, as issues in model prediction may make our project a source of misinformation.
   1.2. This is especially important as we could potentially come across sensitive topics such as political, gender, and racial topics that are already subject to a lot of misinformation.

1.3. Benchmark of accuracy for claim verification is hard to quantify and subject to nuance

2. Quality of data for training the model
   2.1. The data that the model is trained on must be complete, accurate, and representative. Mislabeling data (e.g. labelling a fake news article as “true”) will have unfavourable effects on our model performance.
3. Usage of ongoing events
   3.1. The model will have difficulty in dealing with ongoing events, as our model will not always have the most up-to-date data. Dealing with this is incredibly important as traffic to the extension will increase during ongoing events.
4. Usage on sensitive topics
   4.1. It is important for our model to remain objective and neutral, while also communicating to our users in a non-discriminatory way.
5. Transparency
   5.1. Users must understand why content is flagged in a certain way and be clearly informed about the reasons. Failure to do this will cause distrust of the project.
6. Evaluation
   6.1. It may be difficult to test if the model is “correct”. For example, some articles may not fit strictly in a specific political leaning, or some articles may contain both truth and falsehoods.
7. Wide topics covered
   7.1. The model must be trained on a large dataset of varying topics. Niche topics may have lower accuracy if not accounted for.

Project Development Plan

Veritas
Version
Group
COMPSCI 4ZP6
Supervisor: Dr. Allison Lahnala

Oct 10, 2025

8. Team Meeting and Communication

The primary meeting and communication platform will be Discord, with Teams reserved for any meetings with Dr. Moradi, Dr. Lahnala - our supervisor, or the TAs. Meetings with Dr. Lahnala will be scheduled on an as-needed basis, while we will be booking at least one time slot to meet with Dr. Moradi each month. The meetings will be held in person at any available group meeting rooms on campus for the first month before we divert to virtual Discord meetings. The meeting times have been set to 3:00-5:00 pm on Mondays and 12:00-2:00 pm on Fridays. In case any team member cannot attend in person, they will have the option of joining virtually. We have facilitated automated attendance and note-taking for the Discord meetings, and a dedicated note-taker for in-person meetings. For the purpose of sharing and working on documents together, a shared Google Drive will be used.

9. Team Member Roles

9.1 Team Roles Overview

Team Member	**Role(s)**
Haneef Taher	Claim Verification Model Lead Developer
Keyu Lin	Program Manager, Claim Verification Model Developer
Madhav Bezawada	Claim Verification Model Developer
Divij Dhiraaj	Bias Detection Model Developer
Louis Doan	Front-End Developer
Mustakim Kazi	Bias Detection Model Lead Developer

9.2 Role Descriptions

1. Program Manager
   1.1. Responsible for planning and organizing the team's work. This includes creating milestones, creating and assigning tasks, and maintaining the GitHub repository to ensure it's up to date. They will also ensure that the team stays on track with deadlines and helps resolve potential blockers.
2. Claim Verification Model Lead Developer
   2.1. Leads the development of the claim verification model by overseeing research efforts, dividing tasks, and guiding dataset selection. Responsible for resolving blockers within the team and making key decisions for the model. In addition to

planning and coordination, the lead will still actively contribute to coding and implementation work.

3. Claim Verification Model Developer
   3.1. Focuses on researching, developing, and improving the model. Core responsibilities include dataset exploration, implementation, experimentation, and evaluation of model performance.
4. Bias Detection Model Lead Developer
   4.1. Leads the development of the bias detection model by overseeing research efforts, dividing tasks, and guiding dataset selection. Responsible for resolving blockers within the team and making key decisions for the model. In addition to planning and coordination, the lead will still actively contribute to coding and implementation work.
5. Bias Detection Model Developer
   5.1. Focuses on researching, developing, and improving the model. Core responsibilities include dataset exploration, implementation, experimentation, and evaluation of model performance.
6. Front-End Developer
   6.1. Designs and implements frontend interfaces for the extensions and webpage. Ensures the systems are accessible, responsive and integrate with the backend.

10. Workflow Plan

Our code repository will be hosted on GitHub, from which we can download and run our extension/website. Basic GitHub rules and guidelines will be followed, such as the remote repository referred to as origin or remote, while code on your local device will be referred to as the local repository.

The main branch will be used to establish an initial skeleton framework before we transition to a main and dev branch setup. The main branch will contain a stable version of our product, while the dev branch will hold the latest changes being worked on by developers and team members. As we progress through the term and begin working on specific features, these should be pushed to their own remote branch after thorough testing. A merge request should then be made to the dev branch, where another team member will review it for any potential issues and to ensure a thorough understanding of the feature. After merging into dev, testing should be done again to make sure there are no bugs or problems.

With such intensive testing, we should ideally create a small test suite, automatic or not, to ensure everything is working. The test suite format will depend on how the app structure looks like(likely PyTest) and will have more details as our product gets developed. This approach makes it easier to test each small change and identify potential issues before they lead to cascading failures that are difficult to backtrack through and fix.

Every 4 weeks(this is tentative), or when there is a big change to the product and the current dev branch is stable, that branch should be pushed to main to mark a “stable release/state” during development.

For development, we will organize tasks and deliverables into sprints and follow an agile development cycle where we tackle either an issue or a general work task(such as setting up a Node React app or researching API implementation, but these could also be added to issues/tasks if wanted). There are two meetings a week, so if we finish early or encounter any issues, we can discuss them immediately in the Discord server, which is our main source of communication. We can also use meetings or Discord to hook in other members to our workflow if you desire their skills or need their help.

Storing data will be a complex topic as we are training multiple ML models and will include storing both training/testing data as well as the models. There is a minimum of 1TB of storage on local devices for uploading and downloading processing materials.

Training ML models is computationally expensive and time-consuming, particularly when using multiple models. We plan to make use of McMaster CAS GPU resources and share time training our AI models while downloading and saving models after they have finished training, and we do not have server time anymore. However, we do not know yet what resources or limits are included with CAS training.

During model training, we will need to evaluate the accuracy and progress of each model using the mentioned evaluation metrics, such as macro-F1. This will be a part of our ML library to calculate evaluation metrics and give feedback to the models. Sklearn and other popular ML libraries have methods that calculate F1 and other metrics.

GitHub issues will be used to create tasks, track who is working on them, and facilitate discussions, all of which can be managed natively within GitHub issues. They will be assigned a priority using tags along with potential topic tags for organization. This is handy for creating a roadmap or Gantt chart for later.

11. Proof of Concept Demonstration Plan

For our proof of concept plan, we plan to deliver a working Chromium extension prototype that enables users to analyze highlighted text or inserting the text link then returns the output from our bias detection model. The extension will be limited to bias detection as a proof of concept, with claim verification and claim extraction features support coming later. Additionally, a straightforward landing page will be created to provide an overview of the extension, as well as display the completed privacy and policy documentation.

The bias detection model itself will be capable of identifying and labeling the underlying ideological leaning (such as left, center, or right) present in the submitted article content. It will generate explicit bias labels and can highlight rationale spans or phrases in the text that signal partisan language or perspective. This not only helps users quickly see the detected slant but also makes the findings more transparent and actionable. The focus for the proof of concept is on demonstrating accurate, explainable bias analysis on live article input, validating both the technical implementation and the core user experience for this primary feature.

12. Technology

Frontend

Our frontend will be developed with *TypeScript* as the programming language, *React* as our frontend framework and *Vite* as our build tool. Styling will be handled with the CSS framework *Tailwind*, alongside pre-built components from *Shadcn/UI* and *Framer Motion* for our animation library. *Tanstack Query* will be used to manage data fetching and caching. *Vitest* will be used for unit testing and coverage, *React Testing Library* for integration testing, and *Playwright* for end-to-end tests. *Figma* will be used for designing and creating UI prototypes.

Web Extension

We will use *WXT* as our web extension framework. We will use it alongside *Typescript* as the programming language and *React* as our frontend framework. The same technologies mentioned in the *Frontend* section will also be used here.

Backend

Our backend will use *Python* as the programming language alongside *FastAPI* as our backend framework. We will use the *BCrypt* library for hashing passwords and the *PyJWT* library to handle JWT tokens. Data will be stored in our *MongoDB* database and interfaced using the *Mongoose* library. For testing and coverage, we will use *Pytest*.

Infrastructure

For infrastructure and deployment, we will use *Vercel*. For automating CI/CD, we will use *GitHub Actions*. We will create branches for each feature and only merge them into the main branch after all tests have passed. Our application will be multi-tiered with a data, logic, and presentation tier.

Machine Learning

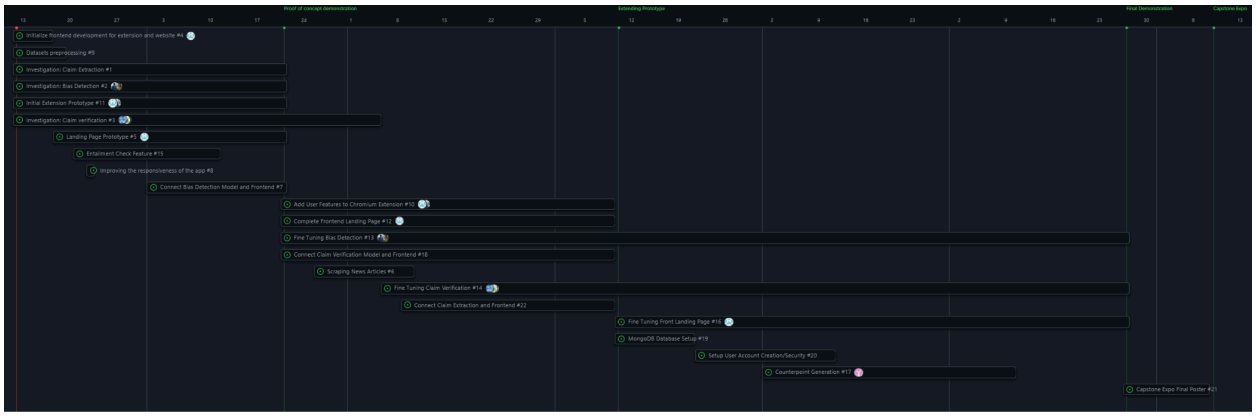
We will use *Python* as the programming language, alongside helpful libraries such as *Pandas* and *NumPy,* for handling matrices and performing mathematical operations. We will use *Matplotlib* and *Seaborn* for the visualization of the data. We will make use of fine-tuned transformers such as *BERT* or *RoBERTa*. A variety of datasets will be used to train our model. McMaster's GPU resources will be used to train our models on a large dataset quickly.

Development Workflow Tools

Our development workflow will use *Git* for version control. We will use *GitHub* for repository management and team collaboration. We will create feature branches and put up a pull request for other group members to review before pushing the change to the main branch.

13. Project Scheduling (Gantt Chart):

This is our Gantt chart and the dates for each task, which we have made through GitHub issues:



1	Initialize frontend development for extension and website #4	Oct 12, 2025	Oct 17, 2025
2	Datasets preprocessing #9	Oct 12, 2025	Oct 19, 2025
3	Investigation: Claim Extraction #1	Oct 12, 2025	Nov 21, 2025
4	Investigation: Bias Detection #2	Oct 12, 2025	Nov 21, 2025
5	Initial Extension Prototype #11	Oct 12, 2025	Nov 21, 2025
6	Investigation: Claim verification #3	Oct 12, 2025	Dec 5, 2025
7	Landing Page Prototype #5	Oct 18, 2025	Nov 21, 2025
8	Entailment Check Feature #15	Oct 21, 2025	Nov 11, 2025
9	Improving the responsiveness of the app #8	Oct 23, 2025	Oct 23, 2025
10	Connect Bias Detection Model and Frontend #7	Nov 1, 2025	Nov 21, 2025
11	Add User Features to Chromium Extension #10	Nov 21, 2025	Jan 9, 2026
12	Complete Frontend Landing Page #12	Nov 21, 2025	Jan 9, 2026
13	Fine Tuning Bias Detection #13	Nov 21, 2025	Mar 27, 2026
14	Connect Claim Verification Model and Frontend #18	Nov 21, 2025	Jan 9, 2026
15	Scraping News Articles #6	Nov 26, 2025	Dec 10, 2025
16	Fine Tuning Claim Verification #14	Dec 6, 2025	Mar 27, 2026
17	Connect Claim Extraction and Frontend #22	Dec 9, 2025	Jan 9, 2026
18	Fine Tuning Front Landing Page #16	Jan 10, 2026	Mar 27, 2026
19	MongoDB Database Setup #19	Jan 10, 2026	Jan 21, 2026
20	Setup User Account Creation/Security #20	Jan 22, 2026	Feb 11, 2026
21	Counterpoint Generation #17	Feb 1, 2026	Mar 10, 2026
22	Capstone Expo Final Poster #21	Mar 27, 2026	Apr 8, 2026